

Protocolo de datagrama de usuario: UDP

Dentro del conjunto de protocolos TCP/IP, el protocolo UDP (Protocolo de Datagrama de Usuario) proporciona mecanismos para que los programas puedan enviar datagramas a otros programas. UDP proporciona puertos que se emplean para distinguir entre múltiples programas que se estén ejecutando en una misma máquina. Por lo tanto, además de los datos enviados, cada mensaje UDP tiene una cabecera que contiene tanto el número de puerto de destino como el número de puerto de origen, haciendo posible que el protocolo UDP en el destinatario envíe el mensaje al destino correcto y al destinatario enviar una respuesta. UDP utiliza el protocolo IP para hacer llegar los mensajes desde un sistema a otro, y proporciona el mismo sistema de envío no fiable y no orientado a la conexión que proporciona IP. No utiliza reconocimientos para asegurar que los mensajes han llegado, no reordena los mensajes recibidos, y no proporciona control de flujo de información. Así, los mensajes UDP pueden perderse, duplicarse, o llegar desordenados.

Identificación del destinatario de un datagrama

La mayoría de los sistemas operativos soportan multiproceso, o lo que es lo mismo, permiten que haya múltiples programas ejecutándose simultáneamente, que denominaremos procesos o tareas. Es evidente que el destinatario último de un mensaje será uno de estos procesos, pero la identificación de uno de estos procesos como destino último de un datagrama tiene cierta complejidad. En primer lugar, los procesos se crean y destruyen de forma dinámica, por ello los emisores del datagramas difícilmente sabrán lo suficiente para identificar un proceso en la máquina de destino. En segundo lugar, los sistemas deben poder reemplazar procesos sin tener que informar a los posibles emisores del datagramas de dichos cambios. Por eso es necesario identificar los destinos según la función que cumplen, y no según el proceso que la implementa. En lugar de pensar en un proceso como el destinatario último de un datagrama, podemos imaginar que cada sistema contiene un conjunto abstracto de puntos de destino denominados Puertos, cada uno de los cuales se identifica mediante un número entero positivo. El sistema operativo proporciona un mecanismo de Interfaces que facilita a los procesos el acceso a los puertos. Para realizar la comunicación, el emisor necesita saber tanto la dirección IP del sistema de destino como el número de puerto del destino dentro de dicho sistema. Cada mensaje contiene tanto el número de puerto de destino en el sistema receptor al cual se envía el mensaje como el número de puerto de origen del sistema emisor al cual deben enviarse

las respuestas; de esta forma, resulta posible para cualquier proceso que reciba un mensaje contestar al emisor.

Formato de los mensajes UDP

Usamos el término datagrama de usuario para describir un mensaje UDP; el énfasis en usuario tiene como objetivo distinguir los datagramas UDP de los datagramas IP. Conceptualmente, un datagrama de usuario consiste en dos partes:

- Un encabezado que contiene meta-información, como los números de puerto de origen y destino
- Un área de datos útil que contiene los datos que se están enviando.

La figura 10.1 ilustra la organización.



Figure 10.1 The conceptual organization of a UDP message.

El encabezado de un datagrama de usuario consiste en cuatro campos que especifican

- El puerto desde el cual se envió el mensaje
- El puerto al cual el mensaje está destinado
- La longitud del mensaje
- Un checksum de UDP

Cada campo tiene una longitud de dieciséis bits, lo que significa que el encabezado completo ocupa un total de solo ocho octetos. La figura 10.2 ilustra el formato del encabezado.

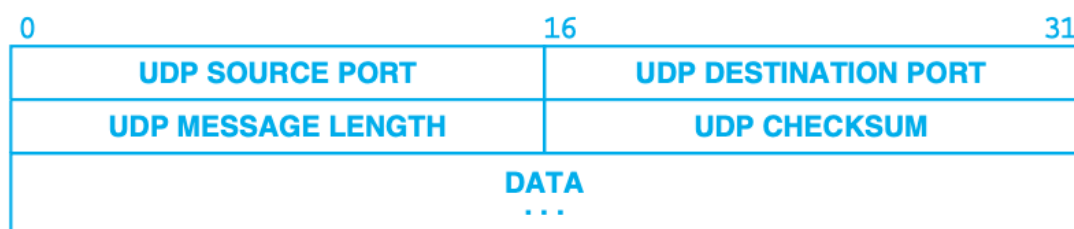


Figure 10.2 The format of fields in a UDP datagram.

El campo UDP SOURCE PORT contiene un número de puerto de 16 bits utilizado por la aplicación emisora, y el campo UDP DESTINATION PORT contiene el número de puerto UDP de 16 bits de la aplicación receptora. UDP SOURCE PORT es opcional. Lo pensamos como identificando el puerto al cual debería enviarse una respuesta. En una transferencia unidireccional donde el receptor no envía una respuesta, como por ejemplo, donde un sensor de temperatura envía cada determinado tiempo valores a un servidor

central, el puerto de origen no es necesario y puede ponerse a cero. El campo UDP MESSAGE LENGTH indica la cantidad de octetos en el datagrama UDP, incluyendo el encabezado UDP y los datos. Por lo tanto, el valor mínimo es ocho, la longitud del encabezado solamente. El campo UDP MESSAGE LENGTH consiste en dieciséis bits, lo que significa que el valor máximo que puede representarse es $2^{16} = 65.535$. Como cuestión práctica, sin embargo, veremos que un mensaje UDP debe caber en el área de carga útil de un datagrama IP. Por lo tanto, el tamaño máximo permitido depende del tamaño del encabezado IP, que son considerablemente más grandes en un datagrama IPv6 que en un datagrama IPv4.

Checksum de UDP

IPv4 e IPv6 difieren en su interpretación del campo CHECKSUM de UDP. Para IPv6, el checksum de UDP es obligatorio. Para IPv4, el checksum de UDP es opcional y no es necesario utilizarlo en absoluto; un valor de cero en el campo CHECKSUM significa que no se ha calculado ningún checksum (es decir, un receptor no debería verificarlo). Los diseñadores de IPv4 eligieron hacer el checksum opcional para permitir que las implementaciones operen con poca sobrecarga computacional al usar UDP a través de una red de área local altamente confiable. Recuerde, sin embargo, que IP no calcula un checksum sobre la porción de datos de un datagrama IP. Por lo tanto, el checksum de UDP proporciona la única forma de garantizar que los datos han llegado intactos y debería utilizarse. A menudo se preguntan qué sucede con los mensajes UDP para los cuales el checksum calculado es cero. Un valor calculado de cero es posible porque UDP utiliza el mismo algoritmo de checksum que IP: divide los datos en cantidades de 16 bits y calcula el complemento a uno de su suma en complemento a uno. El cero no representa un problema porque la aritmética en complemento a uno tiene dos representaciones para el cero: todos los bits puestos a cero o todos los bits puestos a uno. Cuando el checksum calculado es cero, UDP utiliza la representación con todos los bits puestos a uno.

Cálculo del checksum de UDP

1. Parámetros de Entrada

- IP Origen: 192.168.1.1 → 0xC0A80101
- IP Destino: 192.168.1.2 → 0xC0A80102
- Protocolo: UDP (17 en decimal → 0x11)
- Puerto Origen: 5000 → 0x1388
- Puerto Destino: 6000 → 0x1770
- Carga Útil (Payload): "HI" (0x48, 0x49)

- Longitud UDP: 10 bytes (8 bytes de cabecera UDP + 2 bytes de datos) → 0x000A

2. Mapeo de Campos a Palabras de 16 Bits

Componente	Campo	Valor Hexadecimal (16 bits)
Pseudo-cabecera IP	IP Origen (Parte Alta)	0xC0A8
	IP Origen (Parte Baja)	0x0101
	IP Destino (Parte Alta)	0xC0A8
	IP Destino (Parte Baja)	0x0102
	Cero + Protocolo (17)	0x0011
	Longitud UDP	0x000A
Cabecera UDP	Puerto Origen	0x1388
	Puerto Destino	0x1770
	Longitud UDP	0x000A
	Checksum inicial	0x0000
Datos	"HI"	0x4849

3. Suma en Complemento a 1 con Acarreo Circular

Se procede a realizar la suma acumulativa de las palabras de 16 bits. Cada vez que el resultado genera un desbordamiento (acarreo del bit 17), este bit se reinyecta sumándolo al bit menos significativo.

1. $0xC0A8 + 0x0101 = 0xC1A9$
2. $0xC1A9 + 0xC0A8 = 0x18251 \rightarrow$ Acarreo: $0x8251 + 0x0001 = 0x8252$
3. $0x8252 + 0x0102 = 0x8354$
4. $0x8354 + 0x0011 = 0x8365$
5. $0x8365 + 0x000A = 0x836F$
6. $0x836F + 0x1388 = 0x96F7$
7. $0x96F7 + 0x1770 = 0xAE67$
8. $0xAE67 + 0x000A = 0xAE71$
9. $0xAE71 + 0x0000 = 0xAE71$
10. $0xAE71 + 0x4849 = 0xF6BA$

Suma acumulada: 0xF6BA (1111 0110 1011 1010 en binario).

4. Obtención del Checksum

El Checksum final corresponde al complemento a 1 del resultado de la suma (inversión bit a bit de 0xF6BA):

- Suma = $1111\ 0110\ 1011\ 1010_2$
- Complemento a 1 = $0000\ 1001\ 0100\ 0101_2 = \mathbf{0x0945}$

El valor 0x0945 se inserta en el campo Checksum del datagrama UDP antes de ser transmitido.

5. Verificación en el Receptor

El receptor ejecuta el mismo procedimiento sumando las mismas palabras de 16 bits, pero incluyendo el checksum recibido (0x0945) en lugar de 0x0000:

$$\text{Suma de los datos} + \text{Checksum} = 0xF6BA + 0x0945 = \mathbf{0xFFFF}$$

Pseudo-encabezado

El checksum de UDP cubre más información de la que está presente en el datagrama UDP por sí solo. Se extrae información del encabezado IP y el checksum cubre esa información adicional, así como el encabezado UDP y el dato útil. Utilizamos el término pseudo-encabezado para referirnos a la información adicional. Podemos imaginar que el software de checksum de UDP extrae los campos del pseudo-encabezado, los coloca en memoria, agrega una copia del mensaje UDP, y calcula un checksum sobre el objeto completo. Es importante entender que un pseudo-encabezado se utiliza únicamente para el cálculo del checksum. No forma parte del mensaje UDP, no se coloca en un paquete, y nunca se envía a través de una red. El propósito de utilizar un pseudo-encabezado es verificar que un datagrama UDP ha llegado a su destino correcto. La clave para entender el pseudo-encabezado radica en darse cuenta de que el destino correcto consiste en una máquina específica y un puerto específico dentro de esa máquina. El encabezado UDP en sí mismo especifica únicamente el número de puerto. Por lo tanto, para verificar el destino, UDP incluye la dirección IP de destino en el checksum además del encabezado UDP. En el destino final, el software UDP verifica el checksum utilizando la dirección IP de destino obtenida del encabezado del datagrama IP que transportó el mensaje UDP. Si los checksums coinciden, entonces debe ser cierto que el datagrama ha llegado tanto a la máquina de destino prevista como al puerto correcto dentro de esa máquina.

Formato del pseudo-encabezado UDP para IPv4

El pseudo-encabezado utilizado en el cálculo del checksum de UDP para IPv4 consiste en 12 octetos de datos organizados como se ilustra en la Figura 10.3.

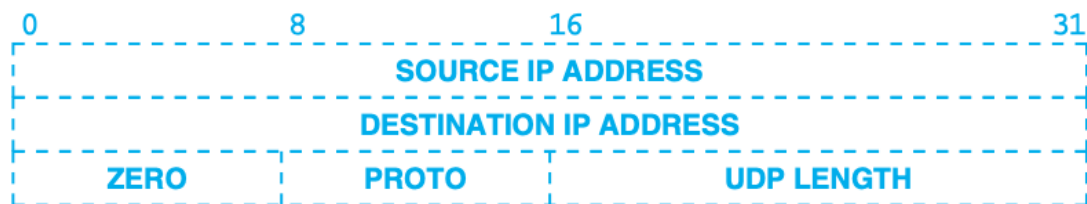


Figure 10.3 The 12 octets of the IPv4 pseudo-header used during UDP checksum computation.

Los campos del pseudo-encabezado etiquetados SOURCE IP ADDRESS y DESTINATION IP ADDRESS contienen las direcciones IPv4 de origen y destino que se colocarán en un datagrama IPv4 al enviar el mensaje UDP. El campo PROTO contiene

el código de tipo de protocolo IPv4 (17 para UDP), y el campo etiquetado UDP LENGTH contiene la longitud del datagrama UDP (sin incluir el pseudo-encabezado). Para verificar el checksum, el receptor debe extraer estos campos del encabezado IPv4, ensamblarlos en el formato de pseudo-encabezado, y calcular el checksum.

Formato del pseudo-encabezado UDP para IPv6

El pseudo-encabezado utilizado en el cálculo del checksum de UDP para IPv6 consiste en 40 octetos de datos organizados como se ilustra en la Figura 10.4.

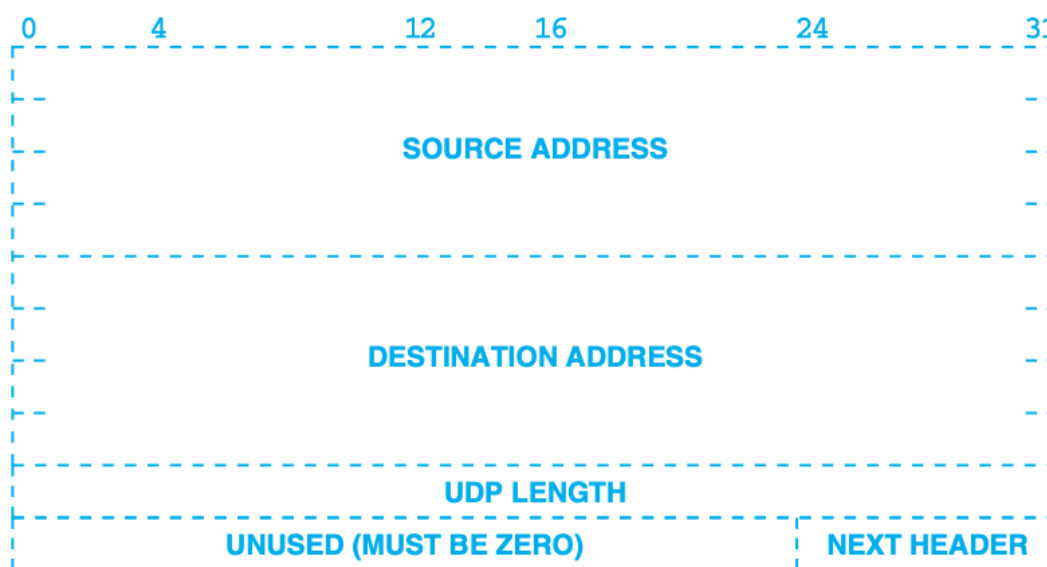


Figure 10.4 The 40 octets of the IPv6 pseudo-header used during UDP checksum computation.

Por supuesto, el pseudo-encabezado para IPv6 utiliza direcciones de origen y destino IPv6. Los otros cambios respecto a IPv4 son que el campo PROTO es reemplazado por el campo NEXT HEADER y que el orden de los campos ha cambiado.

Encapsulado UDP y modelo de capas

UDP proporciona un ejemplo de protocolo de transporte. En el modelo de referencia TCP/IP de 5 capas, UDP se ubica en la capa de transporte por encima de la capa de internet. Conceptualmente, las aplicaciones acceden a UDP, el cual utiliza IP para enviar y recibir datagramas. La figura 10.5 ilustra el modelo de capas conceptual.

Conceptual Layering

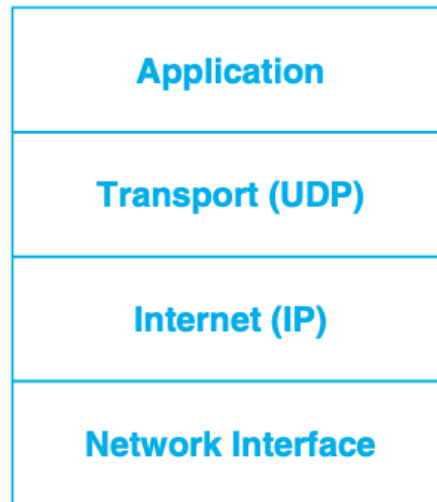


Figure 10.5 The conceptual layering of UDP between application programs and IP.

En el caso de UDP, el modelo de capas conceptual en la figura también implica encapsulado. Es decir, debido a que UDP se ubica en una capa por encima de IP, un mensaje UDP completo, incluyendo el encabezado UDP y la carga de dato útil, se encapsula en un datagrama IP mientras viaja a través de una internet. Por supuesto, el datagrama se encapsula en una trama de red mientras viaja a través de una red subyacente, lo que significa que hay dos niveles de encapsulado. La figura 10.6 ilustra el encapsulado.

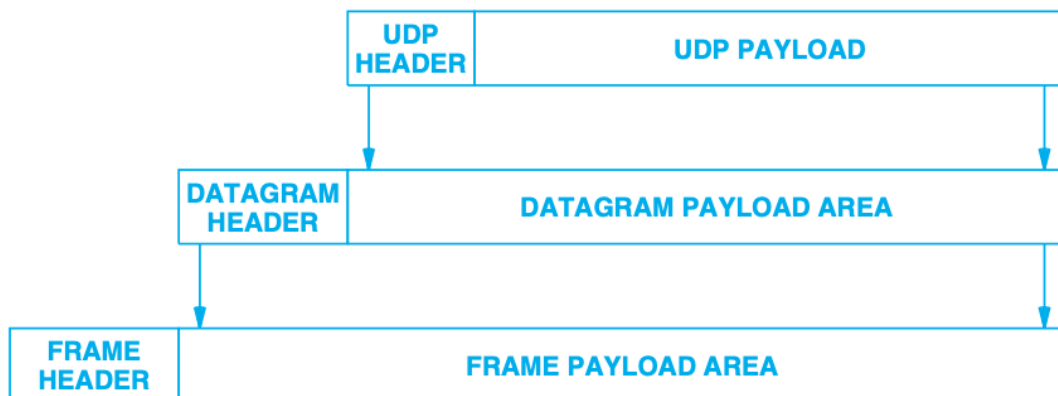


Figure 10.6 Two levels of encapsulation used when a UDP message travels in an IP datagram, which travels in a network frame.

Como la figura indica, el encapsulado resultará en una secuencia lineal de encabezados. Por lo tanto, si se capturara una trama que contuviera UDP, la trama comenzaría con un encabezado de trama seguido de un encabezado IP seguido de un encabezado UDP. En

términos de la construcción de un paquete saliente, podemos imaginar que una aplicación específica datos a enviar. UDP antepone su encabezado a los datos y pasa el datagrama UDP a IP. La capa IP antepone un encabezado IP a lo que recibe de UDP. Finalmente, la capa de red coloca el datagrama IP en una trama antes de enviarlo de una máquina a otra. El formato de la trama depende de la tecnología de red subyacente, pero en la mayoría de las tecnologías una trama incluye un encabezado adicional. El punto es que al observar una trama, el encabezado más externo corresponde a la capa de protocolo más baja, mientras que el encabezado más interno corresponde a la capa de protocolo más alta.

En la entrada, un paquete llega cuando un controlador de dispositivo en la capa de interfaz de red recibe un paquete del dispositivo de interfaz de red y coloca el paquete en memoria. El procesamiento comienza ascendiendo a través de capas sucesivamente más altas del protocolo. Conceptualmente, cada capa elimina un encabezado antes de pasar el mensaje a la capa superior. Para cuando la capa de transporte pasa los datos al proceso receptor, todos los encabezados han sido eliminados.

Al considerar cómo se insertan y eliminan los encabezados, es importante tener presente el principio de modelo de capas. En particular, el principio de modelo de capas se aplica a UDP, lo que significa que el datagrama UDP recibido de IP en la máquina de destino es idéntico al datagrama que UDP pasó a IP en la máquina de origen. Además, los datos que UDP entrega a una aplicación en la máquina receptora serán exactamente los mismos datos que una aplicación pasó a UDP en la máquina emisora.

La división de responsabilidades entre las diversas capas de protocolo es rígida y clara: la capa IP es responsable únicamente de transferir datos entre un par de hosts en una internet, mientras que la capa UDP es responsable únicamente de diferenciar entre múltiples fuentes o destinos dentro de un host. Por lo tanto, solo el encabezado IP identifica los hosts de origen y destino; solo la capa UDP identifica los puertos de origen o destino dentro de un host.

Multiplexado, demultiplexado en UDP, y puertos

El software UDP proporciona ejemplo de multiplexado y demultiplexado.

- El multiplexado ocurre en la salida. En un host determinado, múltiples aplicaciones pueden usar UDP simultáneamente. Así, podemos imaginar el software UDP aceptando mensajes salientes de un conjunto de aplicaciones, colocando cada uno en un datagrama UDP, y pasando los datagramas a IP para su transmisión.

- El demultiplexado ocurre en la entrada. Podemos imaginar UDP aceptando datagramas UDP entrantes de IP, eligiendo la aplicación a la cual el datagrama ha sido enviado, y pasando los datos a la aplicación.

Conceptualmente, todo el multiplexado y demultiplexado entre UDP y las aplicaciones ocurre a través del mecanismo de puertos. En la práctica, cada programa de aplicación debe negociar con el sistema operativo para obtener un número de puerto local y crear los recursos necesarios para enviar y recibir mensajes UDP. Una vez que el sistema operativo ha creado los recursos necesarios, la aplicación puede enviar datos; el código UDP en el sistema operativo creará un datagrama UDP saliente y colocará el número de puerto local en el campo UDP SOURCE PORT. Solo se necesita el número de puerto de destino para manejar el demultiplexado. Cuando procesa un datagrama entrante, UDP acepta el datagrama del software IP, extrae el UDP DESTINATION PORT del encabezado, y pasa los datos a la aplicación. La figura 10.7 ilustra el demultiplexado.

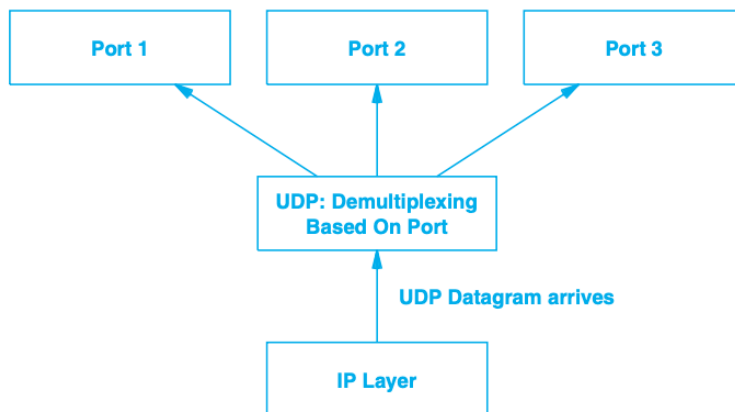


Figure 10.7 Conceptual view of UDP demultiplexing incoming datagrams.

La forma más fácil de pensar en un puerto UDP es como una cola de datagramas entrantes. En la mayoría de las implementaciones, cuando una aplicación negocia con el sistema operativo para asignar un puerto, el sistema operativo crea la cola interna necesaria para retener los datagramas que llegan. La aplicación puede especificar o cambiar el tamaño de la cola. Cuando UDP recibe un datagrama, verifica que el número de puerto de destino coincida con uno de los puertos actualmente en uso. Si encuentra una coincidencia, UDP encola el nuevo datagrama en el puerto donde el programa de aplicación puede acceder a él. Si ninguno de los puertos asignados coincide con el datagrama entrante, UDP envía un mensaje ICMP para informar al origen que el puerto era inalcanzable y descarta el

datagrama. Por supuesto, también ocurre un error si el puerto está lleno. En tales casos, UDP descarta el datagrama entrante y envía un mensaje ICMP.

Números de puerto UDP reservados y disponibles

¿Cómo deberían asignarse los números de puerto? El problema es importante porque las aplicaciones que se ejecutan en dos computadoras necesitan ponerse de acuerdo sobre los números de puerto antes de poder operar. Por ejemplo, cuando un usuario en la computadora A decide realizar una llamada telefónica VoIP a un usuario en la computadora B, el software de aplicación necesita saber qué número de puerto está utilizando la aplicación en la computadora B.

Existen dos enfoques fundamentales para la asignación de puertos. El primer enfoque utiliza una autoridad central. Todos acuerdan permitir que una autoridad central asigne números de puerto según sea necesario y publique la lista de todas las asignaciones. El software se construye de acuerdo con la lista. El enfoque a veces se denomina asignación universal, y las asignaciones de puerto especificadas por la autoridad se denominan asignaciones de puerto bien conocidas (well-known).

El segundo enfoque para la asignación de puertos utiliza la vinculación dinámica. En el enfoque de vinculación dinámica, los puertos no son globalmente conocidos. En cambio, cada vez que un programa de aplicación necesita un número de puerto, el software de protocolo en el sistema operativo elige un número no utilizado y lo asigna a la aplicación. Cuando una aplicación necesita conocer las asignaciones actuales de puertos en otra computadora, la aplicación debe enviar una solicitud que pide una asignación de puerto (por ejemplo, "¿Qué puerto está utilizando el servicio telefónico VoIP?"). La máquina de destino responde dando el número de puerto a utilizar.

Los diseñadores de TCP/IP adoptaron un esquema híbrido que asigna algunos números de puerto a priori, pero deja otros disponibles para que los sitios locales o los programas los asignan dinámicamente. Los números de puerto bien conocidos asignados por la autoridad central comienzan en valores bajos, dejando valores enteros más grandes disponibles para la asignación dinámica. La tabla en la figura 10.8 enumera ejemplos de números de puerto UDP bien conocidos.

Port	Keyword	Description
0	-	Reserved
7	echo	Echo
9	discard	Discard
11	systat	Active Users
13	daytime	Daytime
15	netstat	Network Status Program
17	qotd	Quote of the Day
19	chargen	Character Generator
37	time	Time
42	name	Host Name Server
43	whois	Who Is
53	nameserver	Domain Name Server
67	bootps	BOOTP or DHCP Server
68	bootpc	BOOTP or DHCP Client
69	tfpt	Trivial File Transfer
88	kerberos	Kerberos Security Service
111	sunrpc	ONC Remote Procedure Call (Sun RPC)
123	ntp	Network Time Protocol
161	snmp	Simple Network Management Protocol
162	snmp-trap	SNMP traps
264	bgmp	Border Gateway Multicast Protocol (BGMP)
389	ldap	Lightweight Directory Access Protocol (LDAP)
512	biff	UNIX comsat
514	syslog	System Log
520	rip	Routing Information Protocol (RIP)
525	timed	Time Daemon
546	dhcpv6-c	DHCPv6 client
547	dhcpv6-s	DHCPv6 server
944	nsf	Network File System (NFS) service
973	nfsv6	Network File System (NFS) over IPv6

Figure 10.8 Examples of well-known UDP protocol port numbers.

El Protocolo de Mensajes de Control de Internet (ICMP)

En el sistema no orientado a conexión que hemos descrito hasta ahora, cada router opera de forma autónoma. Cuando llega un datagrama, un router lo reenvía o lo entrega y luego continúa con el siguiente datagrama; el router no coordina con el remitente original de un datagrama. Tal sistema funciona bien si todos los hosts y routers han sido configurados correctamente.

Desafortunadamente, ningún sistema de comunicación de gran tamaño funciona correctamente todo el tiempo. Además de las fallas de hardware de red y de procesador, IP no puede entregar un datagrama si la máquina de destino está temporal o

permanentemente desconectada de la red, si el límite de saltos expira antes de que un datagrama llegue a su destino, o si un router intermedio se sobrecarga tanto que debe descartar un datagrama.

La diferencia importante entre tener una única red implementada con hardware homogéneo y dedicado, y una internet implementada con múltiples sistemas independientes, es que en la primera, el diseñador puede disponer que el hardware subyacente informe a los hosts conectados cuando surgen problemas. En una internet, que no tiene tal mecanismo de hardware, un remitente no puede saber si una falla de entrega resultó de un mal funcionamiento de la red local o de una falla de un sistema en algún lugar del camino hacia un destino. La depuración en tal entorno se vuelve extremadamente difícil.

El propio protocolo IP no contiene nada que ayude al remitente a probar la conectividad o a conocer las fallas. Aunque dijimos que IP no es confiable, queremos que nuestra internet detecte y se recupere de errores siempre que sea posible. Por lo tanto, se necesita un mecanismo adicional.

Para permitir que los routers en internet reporten errores o proporcionen información sobre circunstancias inesperadas, los diseñadores agregaron un mecanismo de propósito especial a los protocolos TCP/IP. Conocido como el Protocolo de Mensajes de Control de Internet (ICMP), el mecanismo se considera una parte requerida de IP.

ICMP está destinado principalmente a informar a una fuente cuando un datagrama enviado experimenta problemas. Sin embargo, el destino final de un mensaje ICMP no es un programa que se ejecuta en la computadora de origen ni el usuario que lanzó la aplicación. En cambio, los mensajes ICMP se envían al software del Protocolo de Internet en la computadora de origen. Es decir, cuando un mensaje de error ICMP llega a una computadora, el módulo de software ICMP en la computadora maneja el mensaje. Por supuesto, ICMP puede tomar acciones adicionales en respuesta al mensaje entrante. Por ejemplo, podría informar a una aplicación o a un protocolo de nivel superior sobre un mensaje entrante.

Podemos resumir: el Protocolo de Mensajes de Control de Internet permite a los routers enviar mensajes de error o control, a la fuente de un datagrama que causó un problema. Los mensajes no se entregan usualmente a las aplicaciones. Pensamos en ICMP como proporcionando comunicación entre un módulo ICMP en una máquina y un módulo ICMP en otra.

ICMP fue diseñado inicialmente para permitir que los routers reporten la causa de errores de entrega a los hosts, pero no está restringido exclusivamente a los routers. Aunque las directrices especifican que algunos mensajes ICMP deberían ser enviados únicamente por routers, una máquina arbitraria puede enviar un mensaje ICMP a cualquier otra máquina. Así, un host puede usarlo para comunicarse con un router u otro host. La principal ventaja de permitir que los hosts lo usen es que proporciona un único mecanismo utilizado para todos los mensajes de control e información.

Reporte de errores vs. corrección de errores

Técnicamente, ICMP es un mecanismo de reporte de errores. Proporciona una forma para que los routers que encuentren un error lo reporten al origen, pero no interactúa con el host ni intenta corregir el error. La idea de reportar problemas en lugar de trabajar para corregirlos surge del principio de diseño fundamental discutido anteriormente: los routers deben ser lo más libres de estado (stateless) posible.

Observamos que la idea de reportar errores en lugar de corregirlos ayuda a mejorar la seguridad. Si un router intentará mantener estado cuando ocurriera un error, un atacante simplemente podría inundar al router con paquetes incorrectos y no responder, o responder muy lentamente, cuando el router intentará corregir el problema. Tal ataque podría agotar los recursos del router. Por lo tanto, la idea de solo reportar errores puede prevenir ciertos ataques de seguridad.

Aunque la especificación del protocolo describe los usos previstos de ICMP y sugiere posibles acciones a tomar en respuesta a los reportes de error, no se especifica completamente la acción a tomar para cada error posible. Por lo tanto, los hosts tienen flexibilidad en cómo relacionan los reportes de error con las aplicaciones. Cuando un datagrama causa un error, ICMP sólo puede reportar la condición de error de vuelta a la fuente original del datagrama; la fuente debe relacionar el error con un programa de aplicación individual o tomar otra acción para corregir el problema.

La mayoría de los errores provienen de la fuente original, pero algunos no. Debido a que ICMP reporta problemas a la fuente original, no puede utilizarse para informar a los routers intermedios sobre problemas. Por ejemplo, supongamos que un datagrama sigue un camino a través de una secuencia de routers, R1, R2, ..., Rk. Si Rk tiene información de enrutamiento incorrecta y envía por error el datagrama al router RE, RE no puede usar ICMP para reportar el error de vuelta al router Rk; ICMP sólo puede enviar un reporte de vuelta a la fuente original. Desafortunadamente, la fuente original no tiene

responsabilidad alguna por el problema y no puede controlar al router que se comporta incorrectamente. De hecho, es posible que la fuente ni siquiera pueda determinar qué router causó el problema.

¿Por qué restringir ICMP a la comunicación con la fuente original? La respuesta debería quedar clara a partir de nuestra discusión sobre los formatos de datagramas y el reenvío. Un datagrama solo contiene campos que especifican la fuente original y el destino final; no contiene un registro completo de su recorrido a través de la internet (excepto en casos inusuales cuando se utiliza la opción de registro de ruta). Además, debido a que los routers pueden establecer y cambiar sus propias tablas de enrutamiento, no existe un conocimiento global de las rutas. Por lo tanto, cuando un datagrama llega a un router determinado, es imposible saber el camino que ha recorrido para llegar. Si el router detecta un problema, IP no puede conocer el conjunto de máquinas intermedias que procesaron el datagrama, por lo que no puede informarlas del problema. En lugar de descartar silenciosamente el datagrama, el router utiliza ICMP para informar a la fuente original que ha ocurrido un problema, y confía en que los administradores de los hosts cooperarán con los administradores de red para localizar y reparar el problema.

Entrega de mensajes ICMP

Los diseñadores adoptaron un enfoque novedoso para el reporte de errores: en lugar de usar un sistema de comunicaciones de nivel inferior para manejar los errores, eligieron usar IP para transportar los mensajes ICMP. Es decir, al igual que todo el resto del tráfico, los mensajes viajan a través de la internet en el área de carga útil de los datagramas IP. Esta elección refleja una suposición importante: los errores son poco frecuentes. En particular, suponemos que el reenvío de datagramas permanecerá intacto la mayor parte del tiempo (es decir, los mensajes de error serán entregados). En la práctica, la suposición ha resultado ser válida: los errores son, en efecto, poco frecuentes. Debido a que cada mensaje ICMP viaja dentro de un datagrama IP, se requieren dos niveles de encapsulado. La figura 9.1 ilustra el concepto.

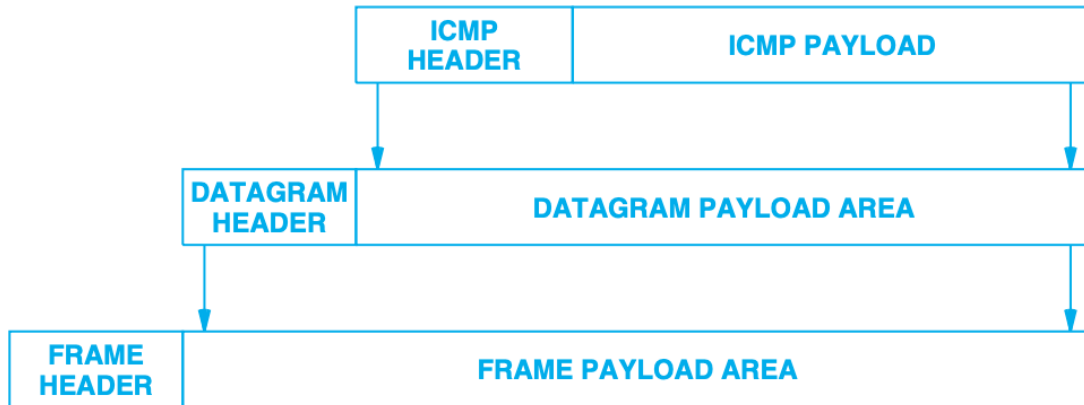


Figure 9.1 The two levels of encapsulation used when an ICMP message is sent across a network.

Como muestra la figura, cada mensaje ICMP viaja a través de una internet en la porción de carga útil de un datagrama IP, el cual a su vez viaja a través de una red subyacente en la porción de carga útil de una trama. Aunque tanto IPv4 como IPv6 utilizan un datagrama para transportar un mensaje ICMP, los detalles difieren. IPv4 utiliza el campo PROTOCOL en el encabezado del datagrama como campo de tipo. Cuando un mensaje ICMP se transporta en el área de carga útil de un datagrama IPv4, el campo PROTOCOL se establece en 1. IPv6 utiliza el campo NEXT HEADER para especificar el tipo del elemento que se transporta. Cuando un mensaje ICMP se transporta en el área de carga útil de un datagrama IPv6, el campo NEXT HEADER del encabezado previo al mensaje ICMP contiene 58.

En términos de procesamiento, un datagrama que transporta un mensaje ICMP se reenvía exactamente igual que un datagrama que transporta información para los usuarios; no existe confiabilidad ni prioridad adicional. Por lo tanto, los propios mensajes de error pueden perderse, duplicarse o descartarse. Además, en una red ya congestionada, el mensaje de error puede aumentar la congestión.

Se hace una excepción a los procedimientos de manejo de errores si un datagrama IP que transporta un mensaje ICMP causa un error. La excepción, establecida para evitar el problema de tener mensajes de error acerca de mensajes de error, especifica que no se generan mensajes ICMP para errores que resulten de datagramas que transportan mensajes de error ICMP.

Modelo de capas conceptual

Habitualmente, el encapsulado y el modelo de capas van de la mano. Por ejemplo, consideremos IP y Ethernet. Cuando viaja a través de una Ethernet, un datagrama IP se

encapsula en una trama Ethernet. El encapsulado sigue el esquema de modelo de capas porque IP es un protocolo de Capa 3 y Ethernet es una tecnología de Capa 2. El campo de tipo de Ethernet permite que una variedad de paquetes de capas superiores se encapsulen en tramas Ethernet sin ambigüedad.

ICMP representa una excepción importante. Aunque cada mensaje ICMP se encapsulan en un datagrama IP, ICMP no se considera un protocolo de nivel superior. En cambio, ICMP es una parte requerida de IP, lo que significa que ICMP se clasifica como un protocolo de Capa 3. Podemos pensar en el encapsulado como el uso del esquema de reenvío existente basado en IP, en lugar de crear un mecanismo de reenvío paralelo para los mensajes ICMP. ICMP debe enviar reportes de error a la fuente original, por lo que un mensaje ICMP debe viajar a través de múltiples redes subyacentes para llegar a su destino final. Por lo tanto, los mensajes ICMP no pueden ser entregados únicamente por un transporte de Capa 2.

Formato de mensaje ICMP

Los estándares definen dos conjuntos de mensajes ICMP: un conjunto para IPv4 y un conjunto más grande para IPv6. En ambas versiones de IP, cada mensaje ICMP tiene su propio formato. Sin embargo, todos los mensajes ICMP comienzan con los mismos tres campos. La figura 9.2 ilustra el formato general de un mensaje ICMP.

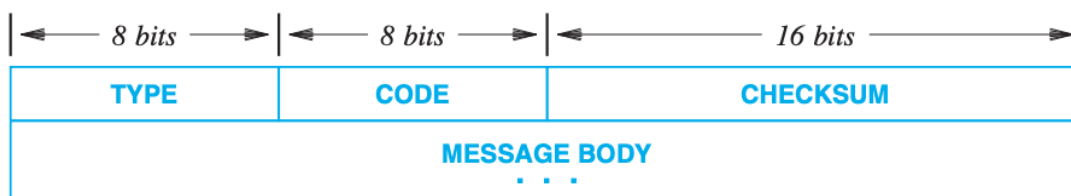


Figure 9.2 The first three fields in each ICMP message.

Como muestra la figura, un mensaje ICMP comienza con un campo TYPE de mensaje ICMP de entero de 8 bits. El campo TYPE identifica el mensaje ICMP específico que sigue. Debido a que el formato de un mensaje está definido por el tipo de mensaje, un receptor utiliza el valor en el campo TYPE para saber cómo interpretar el resto del mensaje.

Un campo CODE de 8 bits en un mensaje ICMP proporciona información adicional sobre el tipo de mensaje. Por ejemplo, un mensaje ICMP TIME EXCEEDED puede tener un valor de código para indicar que el contador de saltos (TTL) del datagrama llegó a cero o que el tiempo de reensamblaje expiró antes de que llegaran todos los fragmentos.

El tercer campo en cada mensaje ICMP consiste en un CHECKSUM de 16 bits que se calcula sobre el mensaje ICMP completo. ICMP utiliza el mismo checksum de complemento a uno de 16 bits que IP.

El cuerpo del mensaje en un mensaje ICMP depende completamente del tipo de ICMP. Sin embargo, para los mensajes ICMP que reportan un error, el cuerpo del mensaje siempre incluye el encabezado más octetos adicionales del datagrama que causó el problema. La razón por la cual ICMP devuelve más que solo el encabezado del datagrama es permitir que el receptor determine con mayor precisión qué protocolo(s) y qué programa de aplicación fueron responsables del datagrama. Como veremos más adelante, los protocolos de nivel superior en la suite TCP/IP están diseñados de manera que la información crucial se codifica en los primeros octetos posteriores al encabezado IP.

Ejemplos de tipos de mensaje ICMP utilizados con IPv4 e IPv6

La figura 9.3 enumera ejemplos de tipos de mensaje ICMP utilizados con IPv4. Las secciones siguientes describen los significados con mayor detalle y proporcionan ejemplos de formatos de mensaje.

Type	Meaning	Type	Meaning
0	Echo Reply	17	Address Mask Request
3	Destination Unreachable	18	Address Mask Reply
4	Source Quench	30	Traceroute
5	Redirect (change a route)	31	Datagram Conversion Error
6	Alternate Host Address	32	Mobile Host Redirect
8	Echo Request	33	Where-Are-You (for IPv6)
9	Router Advertisement	34	I-Am-Here (for IPv6)
10	Router Discovery	35	Mobile Registration Request
11	Time Exceeded	36	Mobile Registration Reply
12	Parameter Problem	37	Domain Name Request
13	Timestamp Request	38	Domain Name Reply
14	Timestamp Reply	39	SKIP (Simple Key Mgmt)
15	Information Request	40	Photuris
16	Information Reply	41	Experimental Mobility

Figure 9.3 Example ICMPv4 message types and the meaning of each.
Values not listed are unassigned or reserved.

Como muestra la figura, muchos de los mensajes ICMP originales fueron diseñados para transportar información en lugar de mensajes de error (por ejemplo, un host utiliza el tipo 17 para solicitar la máscara de dirección utilizada en una red y un router responde con el tipo 18 para reportar la máscara de dirección). IPv6 distingue entre mensajes de error y mensajes informativos dividiendo los valores de tipo en dos conjuntos: los tipos menores

que 128 se utilizan para mensajes de error, y los tipos entre 128 y 255 se utilizan para mensajes informativos. La figura 9.4 enumera ejemplos de tipos de mensaje ICMP utilizados con IPv6 y muestra que, aunque solo se han definido cuatro mensajes de error, IPv6 define muchos mensajes informativos.

Type	Meaning	Type	Meaning
1	Destination Unreachable	138	Router Renumbering
2	Packet Too Big	139	ICMP Node Info. Query
3	Time Exceeded	140	ICMP Node Info. Response
4	Parameter Problem	141	Inverse Neighbor Solicitation
128	Echo Request	142	Inverse Neighbor Advertise.
129	Echo Reply	143	Multicast Listener Reports
130	Multicast Listener Query	144	Home Agent Request
131	Multicast Listener Report	145	Home Agent Reply
132	Multicast Listener Done	146	Mobile Prefix Solicitation
133	Router Solicitation (NDP)	147	Mobile Prefix Advertisement
134	Router Advertise. (NDP)	148	Certification Path Solicitation
135	Neighbor Solicitation (NDP)	149	Certification Path Advertise.
136	Neighbor Advertise. (NDP)	151	Multicast Router Advertise.
137	Redirect Message		

Figure 9.4 Example ICMPv6 message types and the meaning of each.

Como muestra la figura, IPv6 incorpora tres subsistemas principales en ICMP: el Protocolo de Descubrimiento de Vecinos, el soporte de multidifusión, y la movilidad IP. Se han definido mensajes ICMP para cada uno de estos subsistemas. Por ejemplo, al utilizar el Descubrimiento de Vecinos, un nodo IPv6 puede difundir un Mensaje de Solicitud de Vecino (tipo 135) para descubrir vecinos directamente alcanzables, o un Mensaje de Solicitud de Router (tipo 133) para descubrir routers directamente alcanzables.

Ping

El programa ping es quizás la herramienta de diagnóstico de internet más ampliamente utilizada. Originalmente creado para IPv4, ping ha sido extendido para adaptarse a IPv6. En cualquier caso, ping envía un mensaje ICMP Echo Request a una computadora remota. Cualquier computadora que reciba un ICMP Echo Request crea un ICMP Echo Reply y devuelve la respuesta al remitente original. Así, el programa ping recibe el Echo Reply de la máquina remota. El mensaje de solicitud contiene una sección opcional para datos, y la respuesta contiene una copia de los datos que fueron enviados en la solicitud.

¿Cómo puede un simple intercambio de mensajes ayudar a diagnosticar problemas de internet? Al enviar una solicitud de eco, un usuario debe especificar un destino. La

respuesta directa es que una solicitud de eco y su respuesta asociada pueden usarse para probar si un destino es alcanzable y está respondiendo. Debido a que tanto la solicitud como la respuesta viajan en datagramas IP, recibir una respuesta de una máquina remota verifica que las piezas principales del sistema de transporte IP están funcionando correctamente. Primero, el software IP en la computadora de origen debe tener una entrada en su tabla de reenvío para el destino. Segundo, la computadora de origen ha creado un datagrama correcto. Tercero, la fuente pudo alcanzar un router, lo que significa que ARP (IPv4) o el Descubrimiento de Vecinos (IPv6) están funcionando. Tercero, los routers intermedios entre el origen y el destino deben estar operando, y deben reenviar los datagramas correctamente en ambas direcciones entre el origen y el destino. Finalmente, la máquina de destino debe estar en funcionamiento, el controlador del dispositivo debe ser capaz de recibir y enviar paquetes, y tanto los módulos de software ICMP como IP deben estar funcionando.

Existen varias versiones de ping. La mayoría incluye opciones que permiten a un usuario especificar si enviar una solicitud y esperar una respuesta, enviar varias solicitudes y luego esperar una respuesta, o enviar solicitudes periódicamente (por ejemplo, cada segundo) y mostrar todas las respuestas. Si ping envía una serie de solicitudes, muestra estadísticas sobre la pérdida de mensajes. La ventaja de enviar una serie continua de solicitudes surge de la capacidad de descubrir problemas intermitentes. Por ejemplo, considere una red inalámbrica donde la interferencia eléctrica causa pérdidas, pero la interferencia ocurre de forma aleatoria (por ejemplo, cuando una impresora se enciende). La mayoría de las versiones de ping también permiten a un usuario especificar la cantidad de datos que se envían en cada solicitud. Enviar un paquete de ping grande es útil para probar la fragmentación y el reensamblado. Los paquetes grandes también obligan a IPv6 a realizar el descubrimiento de MTU de ruta. Por lo tanto, una aplicación aparentemente trivial tiene varios usos.

Formato de mensaje de solicitud y respuesta de eco

Tanto IPv4 como IPv6 utilizan un único formato para todos los mensajes ICMP Echo Request y Echo Reply. La figura 9.5 ilustra el formato del mensaje.

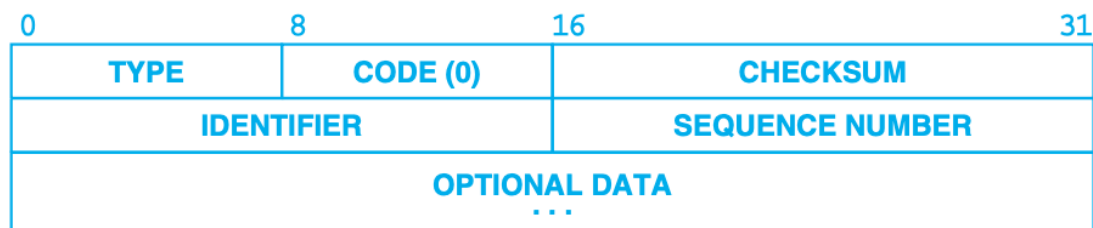


Figure 9.5 ICMP echo request or reply message format.

Aunque el mismo formato de mensaje se utiliza para las solicitudes y respuestas de eco, el valor de TYPE difiere. Para IPv4, el TYPE es 8 en una solicitud y 0 en una respuesta. Para IPv6, el TYPE es 128 en una solicitud y 129 en una respuesta. Para cualquier valor en el campo TYPE, el CODE es cero (es decir, las solicitudes y respuestas de eco no utilizan el campo de código).

Los campos IDENTIFIER y SEQUENCE NUMBER son utilizados por el remitente para hacer coincidir las respuestas con las solicitudes. Un ICMP receptor no interpreta estos dos campos, pero devuelve los mismos valores en la respuesta que se encontraron en la solicitud. Por lo tanto, una máquina que envía una solicitud puede establecer el campo IDENTIFIER a un valor que identifica una aplicación, y puede usar el campo SEQUENCE NUMBER para numerar las solicitudes sucesivas enviadas por la aplicación. Por ejemplo, el IDENTIFIER podría ser el ID de proceso de la aplicación emisora, lo que permite que el software ICMP haga coincidir las respuestas entrantes con la aplicación que envió una solicitud.

El campo etiquetado OPTIONAL DATA es un campo de longitud variable que contiene datos a devolver al remitente. Una respuesta de eco siempre devuelve exactamente los mismos datos que fueron recibidos en la solicitud. Aunque pueden enviarse datos arbitrarios, los programas ping típicos almacenan valores secuenciales en los octetos del área de datos, lo que facilita verificar que los datos devueltos son exactamente los mismos que los datos enviados sin necesidad de almacenar copias de los paquetes. Como se mencionó anteriormente, el tamaño variable permite que un administrador pruebe la fragmentación.

Cálculo del checksum y el pseudo-encabezado de IPv6

Tanto IPv4 como IPv6 utilizan el campo CHECKSUM en un mensaje ICMP, y ambos requieren que el remitente calcule un checksum de complemento a uno de 16 bits del mensaje completo. Además, ambas versiones requieren que el receptor valide el

checksum y descarte los mensajes ICMP que tengan un checksum inválido. Sin embargo, los detalles del cálculo del checksum difieren porque IPv6 agrega un requisito adicional: el checksum utilizado con IPv6 también cubre campos del encabezado base de IP. Conceptualmente, los campos de encabezado designados se organizan en un pseudo-encabezado como se ilustra en la figura 9.6.

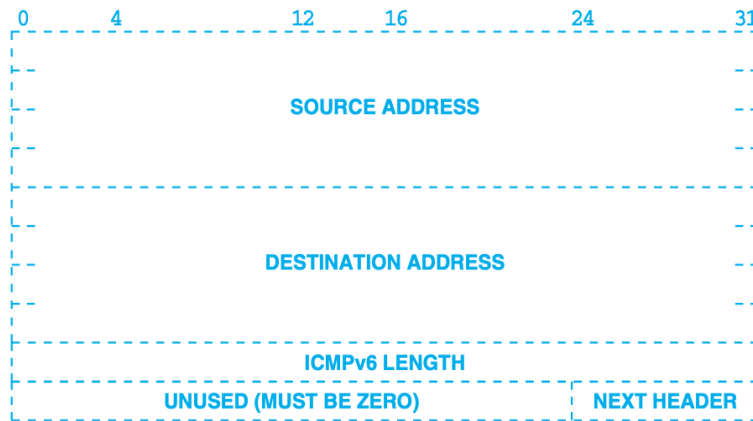


Figure 9.6 The IPv6 pseudo-header used for the checksum computation with ICMPv6.

El término pseudo-encabezado y el uso de líneas discontinuas en la figura indican que la disposición de los campos adicionales se utiliza únicamente para el cálculo del checksum y nunca se coloca en un paquete. Podemos imaginar, por ejemplo, que el código del checksum crea un pseudo-encabezado en memoria copiando campos del encabezado base, agrega una copia del mensaje ICMP al pseudo-encabezado, y luego calcula un checksum a través tanto del pseudo-encabezado como del mensaje.

¿Por qué incluyó IPv6 un pseudo-encabezado en el checksum? Los diseñadores de IPv6 fueron conscientes de posibles debilidades de seguridad, y querían asegurar que una computadora no procesa por error un mensaje ICMP que no estaba destinado a ella. Incluir un pseudo-encabezado en el checksum agrega una verificación adicional de que el mensaje fue entregado correctamente. El pseudo-encabezado no garantiza la corrección, si se necesita una seguridad más fuerte, el datagrama debe cifrarse.

Informes de destinos inalcanzables

Aunque IP implementa un mecanismo de entrega de mejor esfuerzo, descartar datagramas no debe tomarse a la ligera. Cada vez que un error impide que un router reenvíe o entregue un datagrama, el router envía un mensaje ICMP de destino inalcanzable de vuelta a la fuente y luego descarta el datagrama. Los errores de red inalcanzable implican fallas de reenvío en puntos intermedios; los errores de host inalcanzable implican fallas de entrega

en el último salto. Tanto IPv4 como IPv6 utilizan el mismo formato para los mensajes de destino inalcanzable. La figura 9.7 ilustra el formato.

0	8	16	31
TYPE	CODE	CHECKSUM	
UNUSED (MUST BE ZERO)			
PREFIX OF DATAGRAM THAT CAUSED THE PROBLEM			
...			

Figure 9.7 ICMP destination unreachable message format.

Aunque utilizan el mismo formato de mensaje, la forma en que IPv4 e IPv6 interpretan los campos del mensaje difiere ligeramente. IPv4 establece el TYPE en 3, e IPv6 establece el TYPE en 1. Al igual que con los mensajes Echo Request y Reply, IPv4 calcula un CHECKSUM únicamente sobre el mensaje ICMP, e IPv6 incluye un pseudo-encabezado en el checksum. El campo CODE contiene un entero que describe con más detalle el problema; los códigos para IPv4 e IPv6 difieren. La figura 9.8 enumera el significado de los valores de CODE.

IPv4 interpretation

Code	Meaning	Code	Meaning
0	Network unreachable	8	Source host isolated
1	Host unreachable	9	Comm. with net prohibited
2	Protocol unreachable	10	Comm. with host prohibited
3	Port unreachable	11	Net unreachable for TOS
4	Fragmentation needed	12	Host unreachable for TOS
5	Source route failed	13	Communication prohibited
6	Dest. net unknown	14	Host precedence violation
7	Dest. host unknown	15	Precedence cutoff

IPv6 interpretation

Code	Meaning	Code	Meaning
0	No route to dest.	4	Port unreachable
1	Comm. prohibited	5	Source addr. failed policy
2	Beyond src. scope	6	Reject route to dest.
3	Address unreachable	7	Source route error

Figure 9.8 The CODE values for an ICMP destination unreachable message.

Las dos versiones de IP también difieren en la forma en que seleccionan un prefijo del datagrama que causó el problema. IPv4 envía el encabezado del datagrama más los primeros 64 bits de la carga útil del datagrama. IPv6 permite que el datagrama que

transporta el mensaje ICMP tenga hasta 1280 octetos de longitud (la MTU mínima de IPv6), y elige un tamaño máximo de prefijo en consecuencia.

Debido a que el mensaje de error ICMP contiene un breve prefijo del datagrama que causó el problema, la fuente sabrá exactamente qué dirección es inalcanzable. Los destinos pueden ser inalcanzables porque el hardware está temporalmente fuera de servicio, porque el remitente especificó una dirección de destino inexistente, o (en circunstancias poco frecuentes) porque el router no tiene una ruta hacia la red de destino. Observe que aunque los routers reportan las fallas que encuentran, es posible que no detecten todas las fallas de entrega. Por ejemplo, si la máquina de destino se conecta a una red Ethernet, el hardware de red no proporciona confirmaciones. Por lo tanto, un router puede continuar enviando paquetes a un destino después de que el destino se haya apagado, sin recibir ninguna indicación de que los paquetes no están siendo entregados. Para resumir: aunque un router envía un mensaje de destino inalcanzable cuando encuentra un datagrama que no puede ser reenviado o entregado, un router no puede detectar todos esos errores.

El significado de los mensajes de puerto inalcanzable quedará claro cuando estudiemos cómo los protocolos de nivel superior utilizan puntos de destino abstractos llamados puertos. Muchos de los códigos restantes se explican por sí mismos. Por ejemplo, un sitio puede optar por restringir ciertos datagramas entrantes o salientes por razones administrativas. En IPv6, algunas direcciones se clasifican como locales al sitio (site-local), lo que significa que la dirección no puede utilizarse en la Internet global. Intentar enviar un datagrama que tenga una dirección de origen local al sitio activará un error indicando que el datagrama ha sido enviado más allá del alcance válido de la dirección de origen.

Informes de error ICMP relacionados con la fragmentación

Tanto IPv4 como IPv6 permiten que un router reporte un error cuando un datagrama es demasiado grande para una red por la cual debe viajar y no puede fragmentarse. Sin embargo, los detalles difieren. IPv4 envía un mensaje de destino inalcanzable con el campo CODE establecido en 4, e IPv6 envía un mensaje de paquete demasiado grande, que tiene un campo TYPE de 2.

Puede parecer que no se necesitan informes de fragmentación para IPv4 porque un router puede fragmentar un datagrama IPv4. Recuerde que un encabezado IPv4 incluye un bit de "no fragmentar" (do not fragment). Cuando el bit está activado, se prohíbe a un router

realizar la fragmentación, lo que hace que el router envíe un mensaje ICMPv4 de destino inalcanzable con CODE 4.

La razón por la cual IPv6 define un mensaje ICMP separado para reportar problemas de fragmentación surge del diseño. A los routers siempre se les prohíbe fragmentar un datagrama IPv6, lo que significa que una fuente debe realizar el descubrimiento de MTU de ruta. Una parte clave del descubrimiento de MTU de ruta implica recibir información sobre la MTU de las redes remotas. Por lo tanto, el mensaje de paquete demasiado grande contiene un campo que un router utiliza para informar a la fuente sobre la MTU de la red que causó el problema. La figura 9.9 ilustra el formato del mensaje.

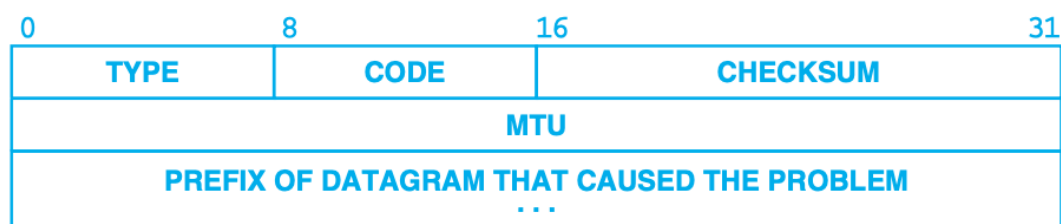


Figure 9.9 The format of an ICMPv6 *packet too big* message.

Solicitudes de cambio de ruta desde los routers

Las tablas de reenvío de los hosts generalmente permanecen estáticas durante largos períodos de tiempo. Un host inicializa su tabla de reenvío al arrancar el sistema, y los administradores del sistema rara vez modifican la tabla durante el funcionamiento normal. Los routers son más dinámicos: intercambian información de enrutamiento periódicamente para adaptarse a los cambios de red y mantener actualizadas sus tablas de reenvío. Por lo tanto, como regla general: se supone que los routers conocen las rutas correctas; los hosts comienzan con información de enrutamiento mínima y aprenden nuevas rutas de los routers.

Para seguir la regla y evitar enviar información de reenvío al configurar un host, la configuración inicial del host generalmente especifica la información de reenvío mínima posible necesaria para comunicarse (por ejemplo, la dirección de un único router predeterminado). Así, un host puede comenzar con información incompleta y depender de los routers para actualizar su tabla de reenvío según sea necesario.

Cuando un router detecta que un host está utilizando un primer salto no óptimo, el router envía al host un mensaje ICMP redirect que le indica que cambie su tabla de reenvío. El router también reenvía el datagrama original hacia su destino. La ventaja del esquema de redirección ICMP es la simplicidad: un host arranca sin necesidad de descargar una tabla

de reenvío y puede comunicarse inmediatamente con cualquier destino. Un router solo envía un mensaje de redirección si el host envía un datagrama por una ruta no preferida. Por lo tanto, la tabla de reenvío del host permanece pequeña, pero tendrá rutas óptimas para todos los destinos en uso.

Debido a que se limitan a interacciones entre un router y un host en una red directamente conectada, los mensajes de redirección no resuelven el problema de propagar información de enrutamiento de una manera general. Para entender por qué, consideremos la figura 9.10, la cual ilustra un conjunto de redes conectadas por routers.

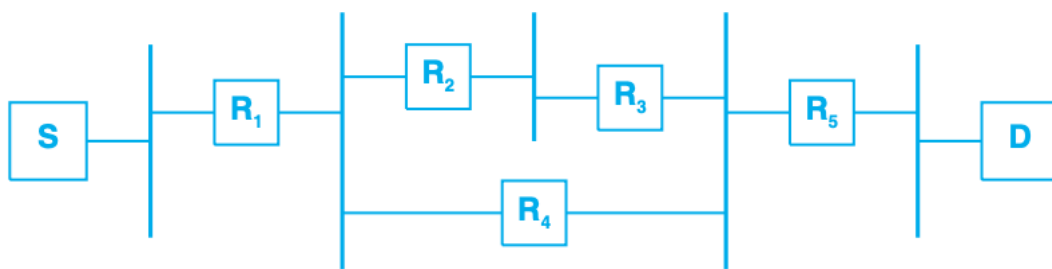


Figure 9.10 Example topology showing why ICMP *redirect* messages do not handle all routing problems.

En la figura, el host S envía un datagrama al destino D. Si el router R1 reenvía incorrectamente el datagrama a través del router R2 en lugar de a través del router R4 (es decir, R1 elige incorrectamente un camino más largo del necesario), el datagrama llegará al router R5. Sin embargo, R5 no puede enviar un mensaje ICMP redirect a R1 porque R5 no conoce la dirección de R1.

Al igual que con varios otros tipos de mensaje ICMP, IPv4 e IPv6 utilizan el mismo formato general para los mensajes de redirección. El mensaje comienza con los campos requeridos TYPE, CODE y CHECKSUM. El mensaje contiene además dos piezas de información: la dirección IP de un router a utilizar como primer salto y la dirección de destino que causó el problema. Los formatos de mensaje difieren. Un mensaje redirect de IPv4 contiene la dirección IPv4 de 32 bits de un router seguida del prefijo del datagrama que fue reenviado incorrectamente. Un mensaje redirect de IPv6 contiene la dirección IPv6 de un router y la dirección IPv6 de destino que debería reenviarse a través del router. La figura 9.11 ilustra el formato utilizado con IPv6.

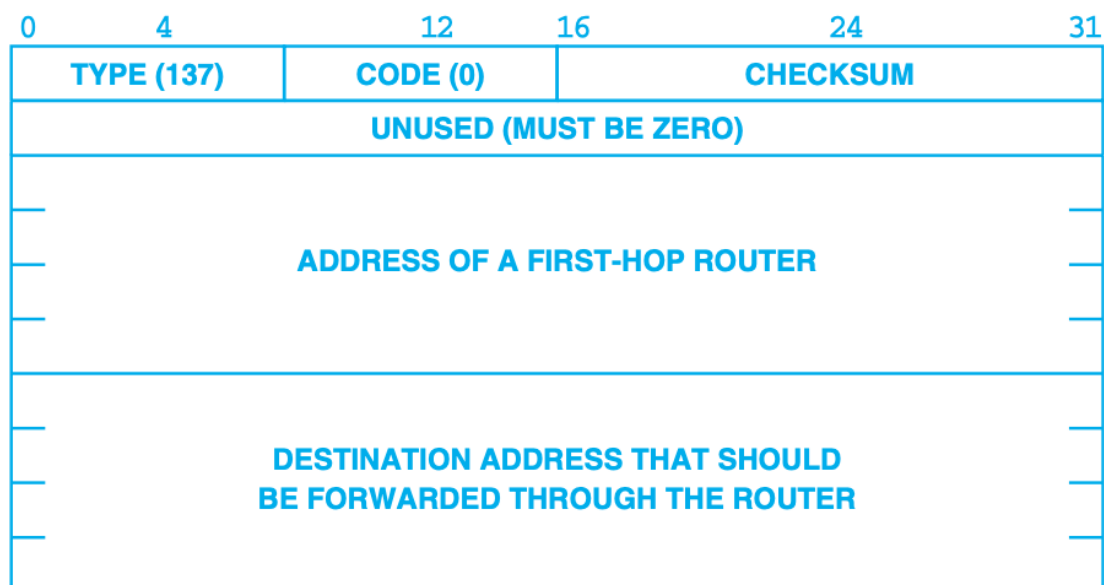


Figure 9.11 The ICMPv6 *redirect* message format.

Como regla general, los routers solo envían solicitudes ICMP redirect a los hosts y no a otros routers.

Detección de rutas circulares o excesivamente largas

Debido a que cada router de internet utiliza información local al reenviar datagramas, los errores o inconsistencias en la información de reenvío pueden producir un ciclo conocido como bucle de enrutamiento (routing loop) para un destino dado, D. Un bucle de enrutamiento puede consistir en dos routers que cada uno reenvía un datagrama dado al otro, o puede consistir en múltiples routers que cada uno reenvía un datagrama al siguiente router del ciclo. Si un datagrama entra en un bucle de enrutamiento, circulará alrededor del bucle indefinidamente.

Para evitar que los datagramas circulen eternamente en una internet TCP/IP, cada datagrama IP contiene un límite de saltos. Cada vez que procesa un datagrama, un router decrementa el límite de saltos y descarta el datagrama cuando el contador llega a cero. Un router no simplemente descarta un datagrama que ha excedido su límite de saltos. En cambio, un router toma la acción adicional de enviar a la fuente un mensaje ICMP time exceeded. Tanto IPv4 como IPv6 envían un mensaje time exceeded, y ambos utilizan el mismo formato que se ilustra en la figura 9.12. IPv4 establece el TYPE en 11, e IPv6 establece el TYPE en 3.

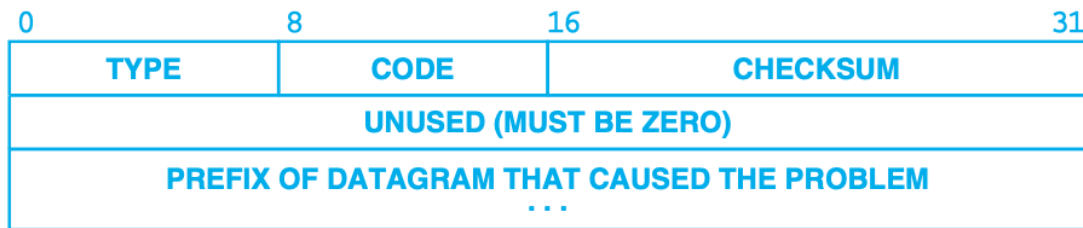


Figure 9.12 ICMP time exceeded message format.

Puede parecer extraño que el mensaje se refiera al tiempo cuando el error que se reporta es un límite de saltos. El nombre se deriva de IPv4, que originalmente interpretaba el límite de saltos como un contador de tiempo de vida (TTL, time to live). Por lo tanto, tenía sentido interpretar la expiración del TTL como la superación de un límite de tiempo. El mensaje time exceeded también se utiliza para reportar otra expiración de tiempo: el tiempo de espera agotado durante el reensamblaje de un datagrama fragmentado. Recuerde que el software IP en un host de destino debe reunir los fragmentos y construir un datagrama completo. Cuando llega un fragmento para un nuevo datagrama, el host establece un temporizador. Si uno o más fragmentos no llegan antes de que el temporizador expire, los fragmentos se descartan y el host receptor envía un mensaje ICMP time exceeded de vuelta a la fuente. ICMP utiliza el campo CODE en un mensaje time exceeded para explicar la naturaleza del tiempo de espera que se reporta, como muestra la figura 9.13.

Code Value	Meaning
0	Hop limit exceeded
1	Fragment reassembly time exceeded

Figure 9.13 Interpretation of the *CODE* field in an ICMP *time exceeded* message. Both IPv4 and IPv6 use the same interpretation.

Informe de otros problemas

Cuando un router o un host encuentra problemas con un datagrama no cubiertos por los mensajes de error ICMP anteriores (por ejemplo, un encabezado de datagrama incorrecto), envía un mensaje parameter problem a la fuente original. En IPv4, una posible causa de tales problemas ocurre cuando los argumentos de una opción son incorrectos. En IPv6, los problemas de parámetros pueden surgir si el valor en un campo de encabezado está fuera de rango, el tipo NEXT HEADER no es reconocido, o una de

las opciones no es reconocida. En tales casos, un router envía un mensaje parameter problem y utiliza el campo CODE para distinguir entre subproblemas. La figura 9.14 ilustra el formato de un mensaje parameter problem. Tales mensajes solo se envían cuando un problema es tan grave que el datagrama debe descartarse.

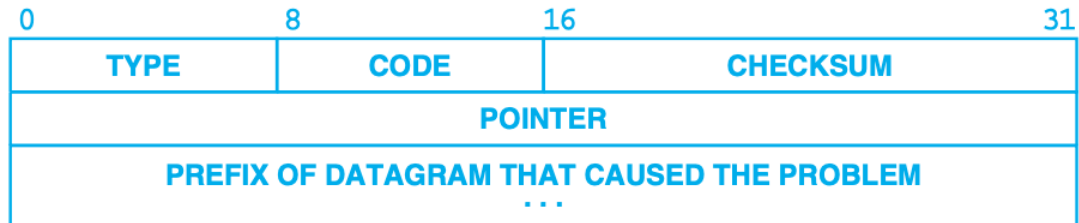


Figure 9.14 ICMP parameter problem message format.

Para hacer que el mensaje no sea ambiguo, el remitente utiliza el campo POINTER en el encabezado del mensaje para identificar el octeto en el datagrama que causó el problema.